PORTFOLIO / TECHNICAL BRIEF

Every event. One accurate bill.

Enterprise Telecommunications Billing & Subscription Management | Angular 22, Java 21, Spring Boot 4.1, PostgreSQL, Kafka and Redis.

VERIFIED LOCAL EXERCISE - 3 OCTOBER 2026

50,000	45,000	5,000	0
Kafka deliveries	unique usage events	intentional duplicates	simulation failures

ARCHITECTURE AND RESPONSIBILITY

Angular customer and administration portals connect to authenticated Spring Boot domain APIs. PostgreSQL stores subscriptions, effective-dated plan segments, accepted producer outbox records, metering receipts, invoices and ledger entries. The outbox publishes to Kafka; a transactional batch consumer commits each unique event and the usage aggregates together.

01	02	03	04
SUBSCRIPTION	**OUTBOX + KAFKA**	**METERING**	**BILLING**
Effective-dated terms	At-least-once transport	Receipts + usage totals	Immutable invoice / ledger

BUSINESS INVARIANTS

Two deduplication keys

A unique delivery receipt prevents repeat processing of one producer message. A unique event ID prevents repeat business consumption.

Billing is not a race

Finalisation refuses to run while an accepted delivery has no durable processing/failure receipt. Closed months reject new usage.

One source of financial truth

Plan prices are snapshotted by service segment; BigDecimal/UTC rules compute charges. Invoice lines and ledger history are protected.

At-least-once delivery

Kafka messages may replay; the database enforces effectively-once business results, not a claim of Kafka-to-SQL exactly-once transactions.

REPOSITORY EVIDENCE

README.md; docs/business-rules.md; docs/verification.md; docs/requirements-signoff.md; backend/src/main/java/com/telecom/{billing,metering,subscription}; backend/src/main/resources/db/migration.

Billing, reliability & limits.

Auditable financial rules, measured recovery checks, and clear separation between implemented features and production integrations.

REPRODUCIBLE OCTOBER 2026 SAMPLE

STARTER 5 GB → EVERYDAY 20 GB

Month-start activation; upgrade on 3 Oct

Usage processed	**45,000 MB**
Pooled prorated allowance	**19,488 MB**
Recurring segments	**ZAR 239.33**
Overage	**ZAR 373.71**
Final invoice	**ZAR 613.04**

SETTLEMENT

The scripted local demo records a simulated payment against this invoice. Retrying the payment request leaves exactly one financial entry, and the due balance becomes ZAR 0.00.

CALCULATION SCOPE

UTC calendar months, 1 GB = 1,024 MB; HALF_UP rounding per line; floor on each segment allowance. Tax, foreign exchange, real network data and real payment gateways are excluded.

VERIFIED RESILIENCE

50,000 + full replay

45,000 unique events retained after a full Kafka replay; consumer group lag drained to zero.

Independent worker

API paused with consumer disabled; separate consumer later drained 900 unique and 100 duplicate events.

Redis outage

Catalogue and entitlement reads fell back to PostgreSQL when Redis was stopped.

Concurrent requests

Four invoice finalisations yielded one invoice; four identical payments yielded one ledger entry.

LIMITATIONS AND HANDOFF

Payments and telecom usage are simulated. Local JWT authentication is implemented; OIDC is documented as a migration path, not a connected provider. The local Kafka setup has a single broker. TLS, external payment and network systems, production secrets, capacity/soak testing, and high availability are future deployment work. The documented GitHub Actions workflow was not observed running remotely in the verification record.